Notes for NN Mechanical Cloaking Project
Rayleigh-wave cloaking: theory, implementation, and loss construction

(project notes)

March 8, 2026

Contents

Chapter 1

Overview and scope

These notes summarize the theoretical and numerical ingredients currently used in the NN mechanical cloaking project: (i) transformation elasticity (with emphasis on the Milton–Briane–Willis framework), (ii) Cosserat (micropolar) elasticity as a practical gauge choice for Rayleigh-wave cloaking, (iii) inverse design options (gradient-free and gradient-based), (iv) weak-form implementation for 2D in-plane elastodynamics (Rayleigh-wave setting) in `JAX-FEM`, (v) absorbing boundary strategies compatible with real-valued solvers (Rayleigh/Caughey damping layers), (vi) reference-field computation templates, and (vii) loss functions for cloaking.

Project code anchor. Implementation details referred to in these notes correspond to the current project file `boundaries.py` (in particular: `RayleighCloakProblem`, `get_tensor_map`, `get_mass_map`, `get_surface_maps`, `cloaking_loss`, and the helper functions for the transformation and symmetrized stiffness construction).

Chapter 2

Transformation elasticity theory (Milton et al.)

2.1 Classical linear elastodynamics in the virtual domain

Let Ω_X be the *virtual* (reference) domain, with coordinates $\boldsymbol{X}$ and displacement $\boldsymbol{U}(\boldsymbol{X}, t)$. In the absence of body forces, the Navier equation reads

$$\nabla_X \cdot \boldsymbol{\sigma}(\boldsymbol{U}) = \rho \ddot{\boldsymbol{U}}, \qquad \boldsymbol{\sigma}(\boldsymbol{U}) = \boldsymbol{C} : \boldsymbol{\varepsilon}(\boldsymbol{U}), \qquad \boldsymbol{\varepsilon}(\boldsymbol{U}) = \tfrac{1}{2}(\nabla_X \boldsymbol{U} + (\nabla_X \boldsymbol{U})^{\mathsf{T}}), \tag{2.1}$$

where $\boldsymbol{C}$ is the (typically symmetric) fourth-order elasticity tensor with minor and major symmetries (for Cauchy elasticity).

2.2 Coordinate transformation and gauge

Let $\boldsymbol{x} = \boldsymbol{\Xi}(\boldsymbol{X})$ be an invertible mapping from virtual to *physical* coordinates. Define the deformation gradient $\boldsymbol{F} = \nabla_X \boldsymbol{x}$ and Jacobian $J = \det \boldsymbol{F}$.

A key point in transformation elastodynamics is that the transformed equation depends on the chosen relation (*gauge*) between virtual and physical displacements. Milton–Briane–Willis (MBW) showed that for general transformations, preserving the form of the governing equations can introduce effective density anisotropy and additional coupling terms (Willis-type constitutive structure).

2.2.1 Cosserat gauge (form invariance with non-symmetric stress)

An alternative gauge is the *identity* gauge

$$\boldsymbol{U}(\boldsymbol{X}, t) = \boldsymbol{u}(\boldsymbol{x}, t), \tag{2.2}$$

which (for 2D in-plane transformations used for carpet cloaks) yields a transformed equation of Navier type:

$$\nabla_x \cdot \left(\boldsymbol{C}^{\text{eff}} : \nabla_x \boldsymbol{u} \right) = \rho^{\text{eff}} \ddot{\boldsymbol{u}}, \tag{2.3}$$

with

$$C^{\text{eff}}_{ijkl} = J^{-1} F_{jJ} F_{lL} C_{iJkL}, \qquad \rho^{\text{eff}} = \rho J^{-1}, \tag{2.4}$$

(using Einstein summation). Note that $\boldsymbol{F}$ acts on the *derivative* indices (j, l), not the DOF indices (i, k), because the coordinate transformation enters through the chain rule $\partial/\partial X_J = F_{jJ}\, \partial/\partial x_j$.

Derivation. Starting from the reference weak form

$$\int_{\Omega_0} C_{IJKL} \frac{\partial u_K}{\partial X_L} \frac{\partial v_I}{\partial X_J} \, d\Omega_0, \tag{2.5}$$

the Cosserat gauge sets $u_i(\boldsymbol{x}) = U_I(\boldsymbol{X})$ with $i = I$. Applying the chain rule $\partial/\partial X_J = F_{jJ}\, \partial/\partial x_j$ and the volume element $d\Omega_0 = d\Omega/J$, we obtain

$$\int_{\Omega} \underbrace{J^{-1} F_{jJ} F_{lL} C_{iJkL}}_{=C^{\text{eff}}_{ijkl}} \frac{\partial u_k}{\partial x_l} \frac{\partial v_i}{\partial x_j} \, d\Omega, \tag{2.6}$$

from which the effective stiffness in **??** is read off. In code (`materials.py`):

```
Cnew = jnp.einsum("jJ,lL,iJkL->ijkl", F, F, C0) / J
```

The resulting $\boldsymbol{C}^{\text{eff}}$ generally preserves *major* symmetries but breaks *minor* symmetries, so that $C^{\text{eff}}_{ijkl} \neq C^{\text{eff}}_{jikl}$ and/or $C^{\text{eff}}_{ijkl} \neq C^{\text{eff}}_{ijlk}$. This is the signature of Cosserat/micropolar-type behavior when expressed as a single fourth-order tensor.

2.3 Triangular carpet transformation (project geometry)

For the project triangular carpet cloak (piecewise linear boundaries), a typical mapping keeps $x_1 = X_1$ and maps $X_2 \in [0, z_2(X_1)]$ to $x_2 \in [z_1(x_1), z_2(x_1)]$ via

$$x_1 = X_1, \qquad x_2 = \left(\frac{z_2(X_1) - z_1(X_1)}{z_2(X_1)} \right) X_2 + z_1(X_1), \tag{2.7}$$

which yields a lower-triangular $\boldsymbol{F}$ (constant F_{22} and piecewise-constant F_{21} for linear z_1, z_2). This is the transformation used in the current project code to build $\boldsymbol{F}(\boldsymbol{x})$ and then $\boldsymbol{C}^{\text{eff}}(\boldsymbol{x})$.

2.4 Symmetrization for near-cloaking

If one wants a standard Cauchy-type symmetric tensor (e.g., to avoid micropolar constitutive laws), a practical approach is to *symmetrize* the Cosserat stiffness. For the 2D in-plane case expressed in an "augmented Voigt" notation (including both 12 and 21 shear components), one can enforce the minor symmetries by arithmetic averaging, chosen such that the elastic energy density for symmetric strains is preserved (near-cloaking rationale). This is the strategy implemented in the project helper `symmetrize_stiffness`.

Chapter 3

Cosserat elasticity (non-symmetric stress)

3.1 Why Cosserat appears in transformation elastodynamics

In the Cosserat gauge (**??**), the transformed PDE is written using $\nabla \boldsymbol{u}$ rather than $\boldsymbol{\varepsilon}(\boldsymbol{u})$. As a result, the stress-like object $\boldsymbol{\sigma} = \boldsymbol{C}^{\text{eff}} : \nabla \boldsymbol{u}$ is not required to be symmetric. This aligns with Cosserat (micropolar) continua, where force stresses can be non-symmetric and couple-stresses may appear in more general formulations.

3.2 Practical consequences for FEM

For a standard displacement-based FEM discretization, the main practical differences are:

- The constitutive mapping is applied to the full gradient $\nabla \boldsymbol{u}$, not only to the symmetric strain.
- The stiffness tensor must be stored/used in a representation that can encode the broken minor symmetries.
- If one replaces $\boldsymbol{C}^{\text{eff}}$ with a symmetrized tensor, the model becomes standard Cauchy elasticity (approximate cloak).

Chapter 4

Inverse design options

This project supports two broad inverse-design routes:

1. **Gradient-free optimization** (robust to non-smoothness, expensive per evaluation).
2. **Gradient-based optimization** (efficient when end-to-end differentiation is stable).

4.1 Gradient-free optimization

4.1.1 Bayesian optimization

Bayesian optimization (BO) models the objective $L(\boldsymbol{\theta})$ using a probabilistic surrogate (often a Gaussian process), and selects the next evaluation using an acquisition function (EI, UCB, etc.). This is attractive when each forward solve is expensive and the parameter dimension is moderate.

4.1.2 Bound-constrained L-BFGS-B (quasi-Newton)

When gradients are available or can be approximated and parameters have bounds, L-BFGS-B is a strong baseline. It is a limited-memory quasi-Newton method using an active-set strategy for box constraints. In practice (e.g., `scipy.optimize.minimize(method='L-BFGS-B')`) it is often competitive even with approximate gradients.

4.2 Gradient-based optimization

4.2.1 End-to-end differentiation

Because `JAX-FEM` is compatible with automatic differentiation (AD), one can differentiate a loss L with respect to:

- geometric parameters of the transformation,
- constitutive parameters (e.g., tensor field parameters, NN weights),
- source parameters, etc.

The key requirement is that the solver path is differentiable (linear solve + assembly + boundary operators). For frequency-domain problems, one typically differentiates through the solution map $\boldsymbol{u}(\boldsymbol{\theta})$ using either implicit differentiation or AD through the linear solver, depending on the implementation.

4.2.2 Stability notes

For cloaking, the loss is often sensitive to: (i) boundary reflections (hence good absorbing layers), (ii) conditioning of the stiffness/mass matrices (anisotropy and strong contrasts), (iii) discretization error near singular transformations (mitigate with offsets / regularization).

Chapter 5

Weak formulation for Rayleigh waves in JAX-FEM

5.1 Governing PDE (frequency domain)

We consider 2D in-plane elastodynamics (plane strain) in the physical domain Ω with coordinates $\boldsymbol{x} = (x_1, x_2)$. A typical frequency-domain form (time-harmonic $e^{-i\omega t}$) is:

$$\nabla \cdot \boldsymbol{\sigma}(\boldsymbol{u}) + \omega^2 \rho \, \boldsymbol{u} = \boldsymbol{f}, \tag{5.1}$$

where $\boldsymbol{\sigma}$ is built either from:

- Cauchy elasticity: $\boldsymbol{\sigma} = \boldsymbol{C} : \boldsymbol{\varepsilon}(\boldsymbol{u})$, or
- Cosserat gauge: $\boldsymbol{\sigma} = \boldsymbol{C}^{\text{eff}} : \nabla \boldsymbol{u}$.

5.2 Weak form

Let $\boldsymbol{v}$ be an admissible test function (respecting Dirichlet constraints). The weak form is:

$$\int_\Omega (\nabla \boldsymbol{v}) : \boldsymbol{C}^{\text{eff}} : (\nabla \boldsymbol{u}) \, d\Omega - \omega^2 \int_\Omega \rho^{\text{eff}} \, \boldsymbol{v} \cdot \boldsymbol{u} \, d\Omega = \int_\Omega \boldsymbol{v} \cdot \boldsymbol{f} \, d\Omega + \int_{\Gamma_N} \boldsymbol{v} \cdot \boldsymbol{t} \, d\Gamma, \tag{5.2}$$

where $\boldsymbol{t}$ is prescribed traction on a Neumann boundary Γ_N. For Rayleigh waves, the free surface typically satisfies traction-free BC: $\boldsymbol{t} = \boldsymbol{0}$.

5.3 Mapping to JAX-FEM hooks

In `JAX-FEM`, a problem class typically provides maps for:

- **Tensor map**: returns the constitutive tensor field (here $\boldsymbol{C}^{\text{eff}}(\boldsymbol{x})$ or symmetrized).

- **Mass map**: returns density-like terms (here $\rho^{\text{eff}}(\boldsymbol{x})$).
- **Surface maps**: implement Neumann/Robin-type integrals, including absorbing boundary contributions.

In the project, these are implemented as:

```
def get_tensor_map(): ...
def get_mass_map(): ...
def get_surface_maps(): ...
```

and used by the `RayleighCloakProblem` class.

Chapter 6

Absorbing boundary conditions (based on absorbing layers)

6.1 Why not PML here

Perfectly matched layers (PML) for elastodynamics often rely on complex coordinate stretching, leading to complex-valued systems. If the framework/toolchain assumes real arithmetic, an alternative is needed.

6.2 Rayleigh/Caughey damping absorbing layers

A simple real-valued absorbing layer strategy is to add Rayleigh (Caughey) damping in a buffer region near truncation boundaries:

$$\boldsymbol{C}'(\boldsymbol{x}) = (1 + i\eta(\boldsymbol{x}))\boldsymbol{C}(\boldsymbol{x}), \qquad \boldsymbol{\rho}'(\boldsymbol{x}) = (1 + i\eta_\rho(\boldsymbol{x}))\rho(\boldsymbol{x}), \tag{6.1}$$

or, equivalently in time domain, introduce damping matrix $\boldsymbol{C}_d = \alpha\boldsymbol{M} + \beta\boldsymbol{K}$. In frequency domain, Rayleigh damping leads to a modified operator:

$$\Big(\boldsymbol{K} + i\omega(\alpha\boldsymbol{M} + \beta\boldsymbol{K}) - \omega^2\boldsymbol{M}\Big)\,\boldsymbol{u} = \boldsymbol{f}. \tag{6.2}$$

The project adopts a *layer profile* $\xi(\boldsymbol{x})$ that increases from 0 (interior) to 1 (outer boundary), and applies damping only in the absorbing layer. This is described and implemented following Semblat et al.

6.3 Implementation sketch (layer profile and boundary hooks)

A typical implementation uses:

- a scalar profile $\xi(\boldsymbol{x}) \in [0, 1]$ in the absorbing region,
- effective $\alpha(\boldsymbol{x}) = \alpha_{\max}\xi(\boldsymbol{x})$, $\beta(\boldsymbol{x}) = \beta_{\max}\xi(\boldsymbol{x})$,
- assembly contributions to the residual/Jacobian consistent with **??**.

In the project code, the key helper is `_xi_profile` and the absorbing contributions are wired through surface maps.

Chapter 7

Reference field computation

7.1 Purpose

A *reference* (target) field is needed to define cloaking losses. Common choices:

- **Pristine medium** (no defect, no cloak): reference Rayleigh wave propagation.
- **Ideal cloak** (Cosserat, non-symmetric tensor): a "best possible" reference if available.

In the current project, the baseline is typically the pristine medium response, compared against the cloaked configuration.

7.2 Practical notes

For stable loss evaluation:

- record reference on the same mesh and at the same frequency ω,
- store displacement components and derived quantities (magnitude, surface trace),
- use consistent normalization (e.g., normalize by reference energy on observation set).

figures/reference_field.png

Figure 7.1: Reference field snapshot (placeholder). Replace with an exported displacement magnitude or component field.

Chapter 8

Loss construction for cloaking

8.1 Observation sets

Define an observation set Γ_{obs} (often a line/strip on the free surface *after* the cloak) and optionally a bulk observation region Ω_{obs} excluding the cloak/defect.

8.2 Canonical loss terms

Let $\boldsymbol{u}(\boldsymbol{x})$ be the field for a candidate cloak, and $\boldsymbol{u}_{\text{ref}}(\boldsymbol{x})$ the reference. Typical losses include:

(1) Field matching on an observation boundary

$$L_\Gamma = \frac{\int_{\Gamma_{\text{obs}}} \|\boldsymbol{u} - \boldsymbol{u}_{\text{ref}}\|^2 \, d\Gamma}{\int_{\Gamma_{\text{obs}}} \|\boldsymbol{u}_{\text{ref}}\|^2 \, d\Gamma + \varepsilon}. \tag{8.1}$$

(2) Bulk matching away from the cloak

$$L_\Omega = \frac{\int_{\Omega_{\text{obs}}} \|\boldsymbol{u} - \boldsymbol{u}_{\text{ref}}\|^2 \, d\Omega}{\int_{\Omega_{\text{obs}}} \|\boldsymbol{u}_{\text{ref}}\|^2 \, d\Omega + \varepsilon}. \tag{8.2}$$

(3) Penalties / regularization Regularize parameters $\boldsymbol{\theta}$ (e.g., smoothness of tensor fields, bounds on anisotropy, etc.):

$$L_{\text{reg}} = \lambda_s \|\nabla \boldsymbol{\theta}\|^2 + \lambda_b \, \text{barrier}(\boldsymbol{\theta}). \tag{8.3}$$

Full loss

$$L(\boldsymbol{\theta}) = w_\Gamma L_\Gamma + w_\Omega L_\Omega + w_{\text{reg}} L_{\text{reg}}. \tag{8.4}$$

8.3 Connection to project implementation

The current project defines a callable `cloaking_loss(...)` that computes a scalar loss based on: (i) a chosen observation set on the free surface, (ii) reference field data, (iii) optional weighting/normalization.

Chapter 9

Sources (selected)

Bibliography

[1] G. W. Milton, M. Briane, and J. R. Willis, "On cloaking for elasticity and physical equations with a transformation invariant form," *New Journal of Physics*, vol. 8, p. 248, 2006. doi:10.1088/1367-2630/8/10/248.

[2] A. N. Norris and A. L. Shuvalov, "Elastic cloaking theory," *Wave Motion*, vol. 48, no. 6, pp. 525–538, 2011. doi:10.1016/j.wavemoti.2011.03.002.

[3] Z. Chatzopoulos, A. Palermo, A. Diatta, S. Guenneau, and A. Marzani, "Cloaking Rayleigh waves via symmetrized elastic tensors," *International Journal of Engineering Science*, vol. 191, 103899, 2023. doi:10.1016/j.ijengsci.2023.103899.

[4] J.-F. Semblat, A. Gandomzadeh, and L. Lenti, "A simple numerical absorbing layer method in elastodynamics," *Comptes Rendus Mécanique*, vol. 338, no. 1, pp. 24–32, 2010. doi:10.1016/j.crme.2009.12.004.

[5] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, "Taking the human out of the loop: A review of Bayesian optimization," *Proceedings of the IEEE*, vol. 104, no. 1, pp. 148–175, 2016. doi:10.1109/JPROC.2015.2494218.

[6] R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu, "A limited memory algorithm for bound constrained optimization," *SIAM Journal on Scientific Computing*, vol. 16, no. 5, pp. 1190–1208, 1995. doi:10.1137/0916069.

[7] *JAX-FEM: Differentiable Finite Element Method in JAX*, GitHub repository (deepmodeling/jax-fem).
`https://github.com/deepmodeling/jax-fem`

[8] *SciPy documentation: L-BFGS-B in `scipy.optimize.minimize`*, SciPy user guide.
`https://docs.scipy.org/doc/scipy/reference/optimize.minimize-lbfgsb.html`